

İçindekiler

Dosya rehberi — her dosya ne işe yarıyor

BÖLÜM 1 — Şu an var olan dosyalar

Proje kökü

_devir/ — iki klasör, aynı içerik

Belgeler — tek tek

BÖLÜM 2 — Depoda olmayan dosyalar

BÖLÜM 3 — Kit nerede durur, projeye ne zaman gelir

★ İKİ AYRI ŞEY — en çok karışan yer

Kurulur kurulmaz kullanabildiğin komutlar

Kitin kendi belgeleri — projene KOPYALANMAZ

★ "Kit kullanım kılavuzu" nerede — zaten sende

/yeni-proje çalışınca projeye YERLEŞECEK dosyalar

Kodun kendisi

Ayarlar ve üretilen dosyalar

Nereye bakmalı — sık sorulanlar

Dosya rehberi — her dosya ne işe yarıyor

Bu dosya "bu da ne" sorusunun tek cevabı. Projedeki her dosya burada açıklanıyor: ne işe yaradığı, kimin okuduğu, elle düzenlenip düzenlenmeyeceği.

⚠ Bugün proje **hazırlık aşamasında** — henüz kod yok. Kod yazılmaya başlandığında yeni klasörler açılacak; onların ne olduğu **BÖLÜM 3**'te önceden anlatıldı.

BÖLÜM 1 — Şu an var olan dosyalar

Proje kökü

Dosya	Ne işe yarıyor	Kim okur	Elle düzenlenir mi
<code>.gitignore</code>	Git'e gönderilmeyecek dosyaların listesi. Ödev metni ve gizli değerler burada listeli	Git	✓
<code>.gitattributes</code>	🚫 Satır sonu kuralı: <code>* text=auto eol=lf</code> . Windows'un <code>CRLF</code> yapmasını engeller — yoksa Linux konteynerindeki betikler çalışmaz	Git	✗
<code>.vscode/extensions.json</code>	VS Code'un "önerilen eklentiler var" uyarısını çıkaran liste. Install All deyince hepsi kurulur	VS Code	✓
<code>.gstack/</code>	Üçüncü taraf bir aracın bıraktığı klasör. 🚫 Projeyle ilgisi yok , silinebilir	—	✗
<code>_devir/</code>	Hazırlık belgelerinin tamamı — aşağıda	İkisi	✓

`_devir/` — iki klasör, aynı içerik

Klasör	İçinde	Ne için
<code>md/</code>	Markdown kaynak dosyaları	VS Code'da okumak, düzenlemek, ajanın okuması
<code>pdf/</code>	Aynı belgelerin PDF hâli	Telefonda okumak

★ Dosya adları birebir aynı — yalnızca uzantı değişir. `md/KURULUM.md` ile `pdf/KURULUM.pdf` aynı belgedir.

⚠️ PDF'lerin hepsi karanlık temalıdır. Yazdırman gerekirse söyle, aydınlık sürüm üretilir — karanlık sürüm sayfayı siyah basar, kartuşu bitirir.

Belgeler — tek tek

YENİ-OTURUM ★

Kim okur: Ajan (yeni Claude Code oturumu)

Projeyi devralan oturumun okuyacağı **ilk ve tek** dosya. İçinde: ajanın rolü ve karakteri · hangi dosyayı hangi sırayla okuyacağı · `/yeni-proje` 'nin soracağı soruların **cevap anahtarı** · sana neyi sorup **neyi sormayacağı** · devralınan **açık işler**.

★ Yeni oturumda yapıştıracağın tek satır:

```
_devir/md/YENİ-OTURUM.md dosyasını oku ve içindeki talimatları uygula.
```

odev

Kim okur: İkisi · 🚫 Depoda YOK

Kurumun verdiği teknik değerlendirme çalışması — **gerçeğin kaynağı**. 32 bölüm.

Sürüm	Ne için
<code>odev.docx</code>	Word aslı. ⚠️ VS Code gösteremez (ikili dosya)
<code>md/odev.md</code>	Okunabilir hâli — %99.8 karakter eşleşmesi doğrulandı
<code>pdf/odev.pdf</code>	Telefonda okumak için

🚫 Üçü de **.gitignore** 'da. Depo herkese açık; kurumun değerlendirme çalışması yayımlanmaz. **Windows makinesine elle taşınmalı** (USB / OneDrive / WhatsApp) — depoyu klonlamak yetmez.

⚠️ Çelişki görürsen **odev.docx** **kazanır**; **.md** yalnızca okuma kolaylığı.

proje-teknoloji-ve-plan ★

Kim okur: İkisi · ~155 sayfa — projenin en büyük belgesi

Üç soruyu birden cevaplıyor:

#	Soru	Nerede
1	Hangi teknoloji seçildi ve neden	BÖLÜM A · C (23 kart)
2	O teknoloji nedir , ne işe yarar	BÖLÜM C · E (kavramlar)
3	Ne sırayla yapılacak	BÖLÜM H (17 adım)

Ayrıca sistemin uçtan uca akışı: **BÖLÜM F** (sunucu tarafı) ve **BÖLÜM G** (arayüz / UI tarafı).

★ Teknik incelemede savunacağın her gerekçe burada. **Tek seferde okunmaz** — başvuru kitabı gibi, aranan bölüme gidilir.

KURUMDAN—OGRENECEKLERİM

Kim okur: Sen

Kuruma soracağın soruların listesi. Her soruda üç şey var: **neden soruyorum** · **cevaba göre ne değişir** · **cevap gelmezse ne yaparım**.

★ **En kritiği §1.1:** Ödev §11 "*kurum tarafından **iletilen** veri tabanı isimlendirme kuralları*" diyor ama **böyle bir doküman ödevin ekinde yok**. Cevap geç gelirse tüm migration'lar yeniden yazılır — ilk gün sorulmalı.

⚠ Cevaplar geldikçe **PRD.md** ve **altyapi-durumu.md** 'ye işlenir; sonra bu dosya silinir.

KURULUM

Kim okur: Sen

Windows makinesinde ne kurulacağı ve nasıl başlanacağı. Üç bölüm: **A** ilk kurulum (programlar, git, GitHub, eklentiler) · **B** bu projeye devam · **C** sonraki projeler.

⚠ İçinde **üç hesabı karıştırma** tablosu var — en sık yapılan hata: kişisel GitHub · belediyenin Claude hesabı · kurumun GitLab'ı.

sunum-anlatim-plani

Kim okur: Sen

Teknik sunumun **anlatım sırası** — 30 dakikalık akış.

🚫 İçinde **bilgi yok, sadece yol tarifi var**. 46 bağlantının hepsi rehbera işaret ediyor. Sebebi: aynı gerekçe iki yerde yazılmaz; biri değişince ikisi birden değişmez.

calisma-kilavuzu

Kim okur: Sen · ⚠ Bu projeye ait değil

Kitle **herhangi bir** projenin nasıl yürütüleceğini anlatıyor: terimler, `/yeni-proje`, ortam değişkenleri, canlıya çıkış yolları, komutlar.

🚫 **Kitten kopyalanır, her projede aynıdır.** Buradan düzeltmek işe yaramaz — bir sonraki projede eski hâliyle geri gelir. Kural değiştirmek için `/kit-senkron`.

DOSYA-REHBERİ

Kim okur: İkisi

Bu dosya. Projeyi ilk kez açan kişi "*bu da ne*" sorusunu buradan cevaplar.

BÖLÜM 2 — Depoda olmayan dosyalar

🚫 Yerelde var, GitHub'a **gönderilmiyor**:

<code>_devir/odev.docx</code>	← kurumun Word dosyası
<code>_devir/md/odev.md</code>	← okunabilir hâli
<code>_devir/pdf/odev.pdf</code>	← telefon için

Sebep: Depo herkese açık. Kurumun değerlendirme çalışması yayımlanmaz — sonraki adaylar bulabilir ve kurum bunu güven ihlali sayabilir.

★ **Rehberdeki alıntılar sorun değil — ölçüldü:** ödevin yalnızca %13'ü rehberde birebir geçiyor (286 cümlelerin 37'si) ve hepsi yorumla birlikte kısa gereksinim satırları. Bu normal **kaynak gösterme**; ödevi yeniden yayımlamak değil.

⚠ Bu üç dosya **git geçmişinden de temizlendi**. Yalnızca `.gitignore`'a eklemek yetmezdi — bir kez commit edilmiş bir dosya geçmişte durur ve `git show <commit>:<dosya>` ile geri alınabilir.

BÖLÜM 3 — Kit nerede durur, projeye ne zaman gelir

★ İKİ AYRI ŞEY — en çok karışan yer

Kiti kurduğunda **iki şey** geliyor ama **aynı anda değil**:

	Komutlar (skill'ler)	Dosyalar
Ne zaman gelir	★ Kurulur kurulmaz	🚫 Yalnızca <code>/yeni-proje</code> çalışınca
Nerede durur	<code>~/.claude/plugins/marketplaces/bariskose-skills/</code>	Senin proje klasöründe
<code>/yeni-proje</code> gerekir mi	🚫 Hayır	✅ Evet

★ Yani `/kit-senkron`, `/video-analiz`, `/pdf-uret` — üçü de kurulumdan hemen sonra çalışır. Bir proje kurmuş olman gerekmiyor.

Kurulur kurulmaz kullanabildiğin komutlar

Komut	Ne yapar
<code>/yeni-proje</code>	Yeni projeyi baştan sona kurar — dosyaları o zaman yerleştirir
<code>/kit-senkron</code>	Projede öğrenilen kuralı kit geri yazar , kitteki iyileştirmeleri projeye getirir
<code>/video-analiz <url></code>	YouTube videosunu inceleyip kitte eksik olanı bulur
<code>/pdf-uret <dosya.md></code>	Markdown'ı karanlık temalı PDF'e çevirir

Kitin kendi belgeleri — projene KOPYALANMAZ

Bunlar kitin kurulduğu klasörde durur, kitin **kendisini** anlatır:

Dosya	Ne anlatır
<code>README.md</code>	Kit ne yapar, hangi komutlar var, klasör yapısı
<code>KURULUM.md</code>	Kit nasıl kurulur, hangi marketplace'ler eklenir

★ "Kit kullanım kılavuzu" nerede — zaten sende

`calisma-kilavuzu` — bu klasörde duruyor ve **kitin kullanım kılavuzu** odur. İçinde: terimler · `/yeni-proje` adımları · ortam değişkenleri · canlıya çıkış yolları · komutların ne zaman kullanılacağı · takılınca ne

yapılacağı.

⚠ **Kitten kopyalanır, her projede aynıdır.** Kit güncellenince yenisi gelir; buradan düzeltmek işe yaramaz.

/yeni-proje çalışınca projeye YERLEŞECEK dosyalar

★ Bunların tam listesi ve ne işe yaradıkları **calisma-kilavuzu** → **BÖLÜM 5'te**. Burada tekrarlanmıyor — kural gereği aynı bilgi iki yerde yazılmaz, biri diğerine işaret eder.

Kısaca ne geliyor:

Grup	Ne	Ayrıntısı
Ajanın kuralları	CLAUDE.md + docs/standards/ (18 dosya)	Kılavuz BÖLÜM 5
Senin belgen	CALISMA-KILAVUZU.md , REPO-YAPISI.md , README.md	Kılavuz BÖLÜM 5
Projeye özel	docs/project/ altında 8 belge (PRD, roadmap, ADR...)	Kılavuz BÖLÜM 5
Ayarlar	.env.example , .env , .vscode/extensions.json	Kılavuz BÖLÜM 5
İzinler	.claude/settings.json — ★ kılavuzda yok, aşağıda	—

★ **.claude/settings.json** ne yapıyor: Ajanın **izin listesi**. **npm run test** , **lint** , **typecheck** gibi zararsız komutları her seferinde sana sormadan çalıştırmasını sağlıyor. Olmasaydı her testte onay isterdi.

Kodun kendisi

Klasör	İçinde ne var
apps/web/	Next.js arayüzü (UI) — 🚫 iş kuralı yok
apps/api/	NestJS API — iş kuralları, transaction, yetki
apps/worker/	Arka plan işleri (BullMQ) — SLA hatırlatma, tarama
packages/contracts/	★ Zod şemaları — web ve api aynı şemayı kullanır
packages/domain/	Saf iş kuralları — 🚫 Prisma/Nest/HTTP bilmez

Ayarlar ve üretilen dosyalar

Dosya	Ne işe yarıyor	Elle düzenlenir mi
<code>.env.example</code>	Hangi ayarların gerektiğinin listesi — değerler boş	✓
<code>.env</code>	Gerçek değerler —  asla commit edilmez	✓
<code>docker-compose.yml</code>	Dört servisi tek komutla ayağa kaldırır	✓
<code>prisma/schema.prisma</code>	Veri modelinin tanımı	✓
<code>prisma/migrations/</code>	Veritabanı değişiklik geçmişi	 Üretilir
<code>docs/database.dbml</code>	Şema dokümanı —  CI, güncel değilse kırmızı yanar	 Üretilir
<code>node_modules/</code>	İndirilen paketler	 Asla

Nereye bakmalı — sık sorularlar

Soru	Cevap
Yeni oturuma nasıl başlarım	<code>md/YENI-OTURUM.md</code> — tek satır yapıştır
Windows'ta ne kuracağım	<code>md/KURULUM.md</code>
Kuruma ne soracağım	<code>md/KURUMDAN-OGRENECEKLERIM.md</code>
Şu teknoloji neden seçildi	<code>md/proje-teknoloji-ve-plan.md</code> → BÖLÜM C
SLA süreleri ne oldu	Aynı dosya → E.4
Sunumda ne anlatacağım	<code>md/sunum-anlatim-planı.md</code>
Kitle nasıl çalışılır	<code>md/calisma-kilavuzu.md</code>
Ödevde ne isteniyor	<code>md/odev.md</code> (depoda yok, elle taşınır)